

# How Tail Recursion and Tail Call Optimization Work

To make use of tail call optimization, a function must use *tail recursion*. In tail recursion, the recursive function call is the last action of a recursive function. In code, this looks like a `return` statement returning the results of a recursive call.

To see this in action, recall the *factorialByRecursion.py* and *factorialByRecursion.html* programs in Chapter 2. These programs calculated the factorial of an integer; for instance,  $5!$  is equivalent to  $5 \times 4 \times 3 \times 2 \times 1$ , or 120. These calculations can be performed recursively because `factorial(n)` is equivalent to `n * factorial(n - 1)`, with the base case of `n == 1` returning 1.

Let's rewrite these programs to use tail recursion. The following *factorialTailCall.py* program has a `factorial()` function that uses tail recursion:

## Python

---

```
def factorial(number, accum=1):
    if number == 1:
        # BASE CASE
        return accum
    else:
        # RECURSIVE CASE
        return factorial(number - 1, accum * number)

print(factorial(5))
```

---

The *factorialTailCall.html* program has the equivalent JavaScript code:

## JavaScript

---

```
<script type="text/javascript">
function factorial(number, accum=1) {
    if (number === 1) {
        // BASE CASE
        return accum;
    }
}
```